Starke Finance Vaults Secure Code Review

Technical Report

StaRKe LLC

02 May 2025
Version: 1.1

Kudelski Security – Nagravision Sàrl

Corporate Headquarters
Kudelski Security – Nagravision Sàrl
Route de Genève, 22-24
1033 Cheseaux sur Lausanne
Switzerland

Confidential

DOCUMENT PROPERTIES

Version:	1.1
File Name:	Kudelski_Security_Starke_LLC_Vault_Secure_Code_Review_v1.1.docx
Publication Date:	02 May 2025
Confidentiality Level:	Confidential
Document Owner:	Jamshed Memon
Document Authors:	Jamshed Memon, Joo Yeon Cho
Document Recipients:	Oscar Garcia Falcon, Sounak Pradhan
Document Status:	Proposal

Copyright Notice

Kudelski Security, a business unit of Nagravision Sàrl, is a member of the Kudelski Group of Companies.
This document is the intellectual property of Kudelski Security and contains confidential and privileged information. The reproduction, modification, or communication to third parties (or to other than the addressee) of any part of this document is strictly prohibited without the prior written consent from Nagravision Sàrl.

TABLE OF CONTENTS

EXECUTIVE SUMMARY 5
1. PROJECT SUMMARY 6
1.1 Context 6
1.2 Scope 6
1.3 Remarks 7
1.4 Additional Note 7
1.5 Follow-up 8
2. STATIC CODE ANALYSIS 9
2.1 Cargo Audit 9
2.2 Npm audit 9
2.3 Cargo clippy 9
2.4 Semgrep 9
3. TECHNICAL DETAILS OF SECURITY FINDINGS 10
3.1 KS-SRK-F-01 Lack of Slippage Protection in `_swap_on_jupiter` 11
3.2 KS-SRK-F-02 Emergency Shutdown Not Present 13
3.3 KS-SRK-F-03 Pyth Price Confidence Interval Not Implemented 14
3.4 KS-SRK-F-04 Duplicate Vault Names and Metadata 16
3.5 KS-SRK-F-05 Potential Underflow 18
3.6 KS-SRK-F-06 Zero Amount Deposit / Withdraw 19
4. OBSERVATIONS 21
4.1 KS-SRK-O-01 Best Secure Code Practice 22
4.2 KS-SRK-O-02 Outdated Dependencies 24
4.3 KS-SRK-O-03 Latency Mitigation 25
4.4 KS-SRK-O-04 Lack of Unit Test Vectors 26
4.5 KS-SRK-O-05 Missing Countermeasure Against Price Manipulation 28
5. METHODOLOGY 29
5.1 Kickoff 29
5.2 Ramp-up 29
5.3 Review 29
5.4 Reporting 30
5.5 Verify 30
6. VULNERABILITY SCORING SYSTEM 31

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

7. CONCLUSION 33
8. REFERENCE 34
9. APPENDIX 35
9.1 Cargo Audit Log 35
9.2 Npm Audit Log 37
DOCUMENT RECIPIENTS 40
KUDELSKI SECURITY CONTACTS 40
DOCUMENT HISTORY 40

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

EXECUTIVE SUMMARY

StaRKe LLC ("the Client") engaged Kudelski Security ("Kudelski", "We") to perform the Starke Finance Vaults Secure Code Review.

The assessment was conducted remotely by the Kudelski Security Team.

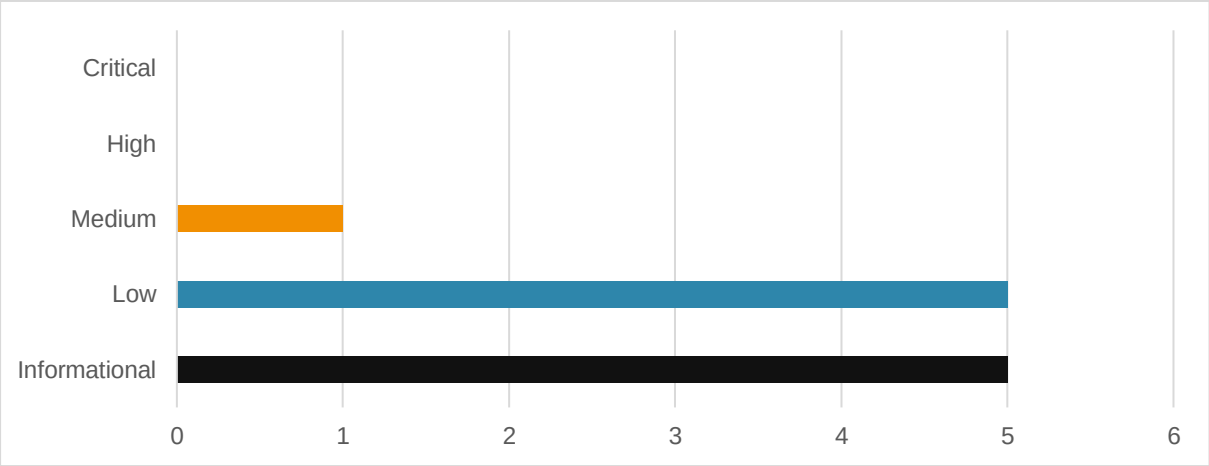
The review took place between 08 April 2025 and 16 April 2025, and focused on the following objectives:

- Provide the customer with an assessment of their overall security posture and any risks that were discovered.
- To provide a professional opinion on the maturity, adequacy, and efficiency of the security measures that are in place.
- To identify potential issues and include improvement recommendations based on the result of our tests.

Key Findings

The following are the major themes and issues identified during the audit period. These, along with other items within the findings section, should be prioritized for remediation to reduce to the risk they pose.

- Lack of Slippage Protection in **`_swap_on_jupiter`**
- Emergency Shutdown Not Present
- Duplicate Vault Names and Metadata



Findings ranked by severity

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

1. PROJECT SUMMARY

This report summarizes the engagement, tests performed, and findings. It also contains detailed descriptions of the discovered vulnerabilities, steps the Kudelski Security Team took to identify and validate each issue, as well as any applicable recommendations for remediation.

1.1 Context

The `vaults` repository contains a secure and flexible Solana program for managing token vaults with advanced features for trustless DeFi applications.

1.2 Scope

The scope consisted in specific Rust files and folders located at:

- https://github.com/starke-labs/vaults

  (commit: `fe6f1b87a02e3e196d8364ecca023814d85d2373`)
- chore: refactor token program usage for vtokens and deposits
- fix: update withdrawal account chunk size from 3 to 4 to accommodate new account structure

The folders and files in scope are:

```
Vaults/programs/vaults/src
        ├── constants.rs
        ├── controllers
        │    ├── deposit_token.rs
        │    ├── mod.rs
        │    ├── token.rs
        │    ├── token_account.rs
        │    ├── token_metadata.rs
        │    ├── token_price.rs
        │    ├── vault_nav.rs
        │    ├── vtoken.rs
        │    ├── whitelist.rs
        │    └── withdraw_token.rs
        ├── instructions
        │    ├── add_token.rs
        │    ├── create_vault.rs
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

```
        │    ├── deposit.rs
        │    ├── initialize_whitelist.rs
        │    ├── mod.rs
        │    ├── remove_token.rs
        │    ├── swap_on_jupiter.rs
        │    ├── update_vault_fees.rs
        │    └── withdraw.rs
        ├── jupiter.rs
        ├── lib.rs
        └── state
            ├── events.rs
            ├── mod.rs
            ├── vault.rs
            └── whitelist.rs
```

The goal of the evaluation was to perform a security audit on the source code.

- No additional systems or resources were in scope for this assessment.
- The dependencies are out of scope of the review.
- Test codes are out of scope.

1.3 Remarks

During the code review, the following positive observations were noted regarding the scope of the engagement:

- The code is well structured.
- Quick and open communication via Teams
- The developers have made a careful and in-depth analysis of their project.
- We had regular and enriching technical exchanges on various topics.

1.4 Additional Note

It is important to notice that, although we did our best in our analysis, no code audit assessment is per se guarantee of absence of vulnerabilities. Our effort was constrained by resource and time limits, along with the scope of the agreement.

In assessing the severity of some of the findings we identified, we kept in mind both the ease of exploitability and the potential damage caused by an exploit.

While assessing the severity of the findings, we considered the impact, ease of exploitability, and the probability of attack. This is a solid baseline for severity determination. Information

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

about the severity ratings can be found in Chapter Vulnerability Scoring System of this document.

1.5 Follow-up

After the initial report (V1.0) was delivered, the Client addressed or acknowledged all vulnerabilities and weaknesses in the following codebase revision:

- fix: security audit findings #4

  (commit: c745be5e4fa25289f802082565cc301eb7127656)

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

2. STATIC CODE ANALYSIS

2.1 Cargo Audit

`Cargo audit` (v0.21.2) identified 3 vulnerabilities and 5 warnings on dependencies. Among those, the vulnerable dependencies are reported in Chapter 3.1. The outputs of cargo audit are in the Appendix 8.

2.2 Npm audit

`npm audit` (v9.5.1) identified 16 vulnerabilities (4 moderate, 12 high) on node modules. The vulnerable dependencies are reported in Chapter 3.1. The outputs of npm audit are in the Appendix 8.

2.3 Cargo clippy

`Cargo clippy` (v0.1.83) identified 28 warnings from the code in scope, which turned out they were false alarms or not notable.

2.4 Semgrep

`Semgrep` (v1.99.0) did not identify any finding in the `program/vaults` folder.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3. TECHNICAL DETAILS OF SECURITY FINDINGS

This chapter provides detailed information on each of the findings, including methods of discovery, explanation of severity determination, recommendations, and applicable references.

The following table provides an overview of the security findings.

#	SEVERITY	TITLE	STATUS
KS-SRK-F-01	Medium	Lack of Slippage Protection in `_swap_on_jupiter`	Acknowledged
KS-SRK-F-02	Low	Emergency Shutdown Not Present	Resolved
KS-SRK-F-03	Low	Pyth Price Confidence Interval Not Implemented	Resolved
KS-SRK-F-04	Low	Duplicate Vault Names and Metadata	Resolved
KS-SRK-F-05	Low	Potential Underflow	Resolved
KS-SRK-F-06	Low	Zero Amount Deposit / Withdraw	Resolved

Findings overview.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3.1 KS-SRK-F-01 Lack of Slippage Protection in `_swap_on_jupiter`

Severity	Impact	Likelihood	Status
Medium	High	Low	Acknowledged

Description

The `_swap_on_jupiter` function does not validate the results of the token swap after invoking the Jupiter program. Specifically, it does not:

- Verify the post-swap balance of the `output_token_account` to ensure the correct amount of tokens was received.
- Check for slippage to ensure the received amount meets the expected minimum output.

Impact

If the swap results in fewer tokens than expected due to slippage or manipulation, the vault could lose value.

Evidence

```
invoke_signed(
    &Instruction {
        program_id: *ctx.accounts.jupiter_program.key,
        accounts,
        data,},
    &accounts_infos,
    signer_seeds,
)?;
```

`Location: programs/vaults/src/instructions/swap_on_jupiter.rs, line 56`

Affected Resources

- `programs/vaults/src/instructions/swap_on_jupiter.rs`

Recommendation

- Reload the `output_token_account` after the swap and calculate the difference between the pre-swap and post-swap balances to determine the amount of tokens received.
- Require a `min_output_amount` parameter and ensure the received amount meets or exceeds this value.

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

Client's Comment

The instruction for the Jupiter swap takes binary data, and we use Jupiter's swap API to get the instruction data from a quote and encode it and send it to our instruction which in turn sends it to the Jupiter's instruction.

We also need the Starke authority's signature to actually run the swap and thus the manager would need to use Starke's API to generate the transaction and have it signed by the Starke authority.

During this off-chain process we can make sure that the transaction only goes through if Jupiter's swap API returns a quote that is below the slippage threshold that we allow.

So, as a conclusion this won't be an on-chain functionality but more of an off-chain functionality and we will make sure that slippage protection is being taken care of.

To make sure that none can game the system the Starke authority's key is necessary along with the manager's key for this transaction.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3.2 KS-SRK-F-02 Emergency Shutdown Not Present

Severity	Impact	Likelihood	Status
Low	Low	Low	Resolved

Description

There are no emergency shutdown or halt functions in the project. In the event of a serious security issue, an emergency shutdown or a pause mechanism would be useful to halt all transactions and prevent additional damage immediately.

Impact

Lack of emergency function could lead to a severe damage to the business. An unexpected security breach may cause the repeated withdrawal of tokens from the vault to the hacker's account, however, such attack may not be stopped immediately due to the lack of shutdown mechanism.

Evidence

No evidence.

Affected Resources

- `programs/vaults/src/controllers/`

Recommendation

Implement an emergency shutdown mechanism which could reduce the risk from a serious security breach. See the Reference below.

Reference

- https://docs.makerdao.com/smart-contract-modules/shutdown

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3.3 KS-SRK-F-03 Pyth Price Confidence Interval Not Implemented

Severity	Impact	Likelihood	Status
Low	Low	Low	Resolved

Description

The Pyth network provides the price with the confidence interval. However, the confidence interval is not implemented in the `get_token_price_from_pyth_feed` function. In certain cases, it would be exposed to a price manipulation attack. Note that this is already noted in the `get_nav` function as TODO.

Impact

The price of tokens can diverge for several reasons, for instance, when a cryptocurrency exchange blocks withdrawals on an asset. Without the usage of the confidence interval, users may not be protected from these unusual market conditions.

Evidence

```
pub fn get_token_price_from_pyth_feed<'info>(
    price_feed_id: &str,
    price_update: &Account<'info, PriceUpdateV2>,
) -> Result<Price> {
    let feed_id: [u8; 32] = get_feed_id_from_hex(&price_feed_id)?;
    let price = price_update.get_price_no_older_than(
        &Clock::get()?,
        PYTH_PRICE_FEED_MAX_AGE_SECONDS,
        &feed_id,
    )?;
    Ok(price)
}
```

Location: programs/vaults/src/controllers/token_price.rs, line 12

```
pub fn get_nav<'info>(
...
// TODO: Throw error if confidence interval is above threshold
//       https://docs.pyth.network/price-feeds/best-practices#confidence-intervals
```

Location: programs/vaults/src/state/vault.rs, line 141

Affected Resources

- `programs/vaults/src`

Recommendation

Consider implementing the confidence interval to determine the range in which the true price probably lies. For example, the following pseudo code might be used for the confidence threshold set to 1%.

```
    const CONFIDENCE_THRESHOLD = 1.0; // 1%
    let confidence = (price.conf * 100.0) / price.price.abs;
    if (confidence > CONFIDENCE_THRESHOLD)  return Err;
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

Reference

- https://www.pyth.network/blog/what-is-confidence
- https://docs.pyth.network/price-feeds/best-practices#confidence-intervals

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3.4 KS-SRK-F-04 Duplicate Vault Names and Metadata

Severity	Impact	Likelihood	Status
Low	Low	Low	Resolved

Description

Currently, managers can create vaults that share identical names, symbols, or URIs. Although each vault's PDA is unique (due to the manager's public key), this duplication can lead to user confusion or even enable phishing attempts if malicious actors create vaults with the same identifiers as legitimate ones. The code also accepts arbitrary strings for `name`, `symbol`, and `uri` without validating their length, format, or content.

Impact

Users might accidentally interact with the wrong vault. Furthermore, malicious vault owners can exploit user trust by mimicking legitimate vaults. There is also potential for misleading or malicious metadata if input is not validated.

Evidence

```
p pub fn _create_vault(
    ctx: Context<CreateVault>,
    name: &str,
    symbol: &str,
    uri: &str,
    entry_fee: u16,
    exit_fee: u16,
) -> Result<()> {
    msg!("Processing vault creation request");
    msg!("Manager: {}", ctx.accounts.manager.key());
    msg!(
        "Deposit token mint: {}",
        ctx.accounts.deposit_token_mint.key()
    );
    msg!("Name: {}", name);
    msg!("Entry fee: {}, Exit fee: {}", entry_fee, exit_fee);

    // Initialize vtoken metadata
    let manager = ctx.accounts.manager.key();
    let vault_seeds = &[Vault::SEED, manager.as_ref(), &[ctx.bumps.vault]];
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

```
    let signer_seeds = &[&vault_seeds[..]];
```

Location: programs/vaults/src/instructions/create_vault.rs, Line 13

Affected Resources

- programs/vaults/src/instructions/create_vault.rs

Recommendation

It is recommended to modify the vault creation process to include additional unique seeds (e.g., vault name) or require a name registry check. This ensures each vault has distinct identifiers, reducing confusion and phishing risks. It is also recommended to add input validation for `name`, `symbol`, and `uri` . For example, the following pseudo code might be used.

```
require!(
    name.len() <= 32 && !name.is_empty(),
    VaultError::InvalidNameLength
);
require!(
    symbol.len() <= 10 && !symbol.is_empty(),
    VaultError::InvalidSymbolLength
);
require!(
    uri.len() <= 200 && !uri.is_empty(),
    VaultError::InvalidUriLength
);
```

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3.5 KS-SRK-F-05 Potential Underflow

Severity	Impact	Likelihood	Status
Low	Low	Low	Resolved

Description

The `_withdraw` function does not verify whether the input parameter `amount` is not larger than the vtoken mint supply.

Impact

This would cause an arithmetic underflow panic since Rust performs checked arithmetic in debug builds and traps on overflow/underflow in release builds.

Evidence

```
pub fn _withdraw<'info>(
    ctx: Context<'_, '_, 'info, 'info, Withdraw<'info>>,
    amount: u64,
) -> Result<()> {
...
    emit!(Withdrawn {
        // TODO: Check if this is correct
        new_vtoken_supply: ctx.accounts.vtoken_mint.supply - amount,
        timestamp: ctx.accounts.clock.unix_timestamp,

    });
```

```
Location: programs/vaults/src/instructions/withdraw.rs, line 10
```

Affected Resources

- `programs/vaults/src/instructions/withdraw.rs`
- `programs/vaults/src/lib.rs`

Recommendation

Add the input validation at the beginning to ensure the withdrawal amount is never larger than the total supply, preventing potential underflows and invalid state changes. For example, the following pseudo code might be used.

```
   require!(
        amount <= ctx.accounts.vtoken_mint.supply,
        WhitelistError::InvalidAmount
    );
```

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

3.6 KS-SRK-F-06 Zero Amount Deposit / Withdraw

Severity	Impact	Likelihood	Status
Low	Low	Low	Resolved

Description

Both `deposit` and `withdraw` functions lack validation for zero amount transactions at both instruction and controller levels. This allows users to execute transactions with amount = 0, which could waste computational resources and pollute the transaction history.

Impact

- Transaction history pollution with zero-amount transactions
- Wasted transaction fees for users
- Potential for spam attacks on the protocol
- Misleading event emissions with zero amounts

Evidence

```
Deposit.rs
pub fn _deposit<'info>(
    ctx: Context<'_, '_, 'info, 'info, Deposit<'info>>,
    amount: u64,  // No validation
) -> Result<()> {

Deposit_token.rs

pub fn calculate_deposit_token_value<'info>(
    whitelist: &Account<'info, TokenWhitelist>,
    deposit_token_mint: Pubkey,
    deposit_token_decimals: u8,
    amount: u64,  // No validation
    price_update: &Account<'info, PriceUpdateV2>,
) -> Result<u64> {

Withdraw.rs
pub fn _withdraw<'info>(
    ctx: Context<'_, '_, 'info, 'info, Withdraw<'info>>,
    amount: u64,  // No validation
) -> Result<()> {

Withdraw_token.rs

pub fn withdraw_all_tokens<'info>(
    // ...
    vtoken_amount: u64,  // No validation
    // ...
) -> Result<()> {
```

```
Location: programs/vaults/src/instructions/withdraw.rs, line 10
  programs/vaults/src/controllers/withdraw_token.rs, line 12
  programs/vaults/src/instructions/deposit.rs, line 13
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

```
programs/vaults/src/controllers/deposit_token.rs, line 10
```

Affected Resources

- `programs/vaults/src/instructions/withdraw.rs`
- `programs/vaults/src/controllers/withdraw_token.rs`
- `programs/vaults/src/instructions/deposit.rs`
- `programs/vaults/src/controllers/deposit_token.rs`

Recommendation

Add the amount validation at the instruction level.

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4. OBSERVATIONS

This chapter contains additional observations that are not directly related to the security of the code, and as such have no severity rating or remediation status summary. These observations are either minor remarks regarding good practice or design choices or related to implementation and performance. These items do not need to be remediated for what concerns security, but where applicable we include recommendations.

#	SEVERITY	TITLE	STATUS
KS-SRK-O-01	Informational	Best Secure Code Practice	Resolved
KS-SRK-O-02	Informational	Outdated Dependencies	Resolved
KS-SRK-O-03	Informational	Latency Mitigation	Informational
KS-SRK-O-04	Informational	Lack of Unit Test Vectors	Informational
KS-SRK-O-05	Informational	Missing Countermeasure Against Price Manipulation	Informational

Observations overview.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.1 KS-SRK-O-01 Best Secure Code Practice

Description

- When the function `checked_mul` is used to calculate the amount or price of token, a conversion to `u128` might be considered to avoid a potential overflow error. Note that some function such as `calculate_vtokens_to_mint` has such conversion.

```
.checked_mul(price_in_nav_decimals)
.checked_mul(withdrawal_ratio)
.checked_mul(PRECISION)
```

```
Location: programs/vaults/src/controllers/vault_nav.rs, line 63,
  programs/vaults/src/controllers/withdraw_token.rs, line 231,
    programs/vaults/src/controllers/token_price.rs, line 26
```

- The order of parameter is not matched with the definition of `TransferChecked` struct: from, mint, to, and authority. Although this does not cause any issue, it would be better to make it consistent to avoid any confusion.

```
let transfer_accounts = TransferChecked {
    from: from.to_account_info().clone(),
    to: to.to_account_info().clone(),
    mint: mint.to_account_info().clone(),
    authority: (*authority).clone(),
};
```

```
Location: programs/vaults/src/controllers/token.rs, line 47
```

- The comment is not matched: should it be 4?

```
// Each chunk must contain 3 accounts in this order:
// 1. Token mint
// 2. Vault token account
// 3. User token account
// 4. Token program
```

```
Location: programs/vaults/src/controllers/withdraw_token.rs, line 116
```

- Since the function accepts even when the user account does not exist, the TODO comment is valid and should be implemented.

```
if chunk[2].data_is_empty() {
    // User token account doesn't exist
    create_associated_token_account(
...
// TODO: What happens if the user token account doesn't exist yet?
msg!("Checking if mint and user token account match");
require!(
    mint.key() == user_token_account.mint,
```

```
Location: programs/vaults/src/controllers/withdraw_token.rs, line 160
```

- The vault supports both SPL token and Token 2022 program. However, it is not explicitly mentioned in the README.md or in-line comments. In comparison, it is commented that vtoken supports only SPL token at the moment, for instance, `programs/vaults/src/instructions/withdraw.rs#L103`.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

Affected Resources

- `programs/vaults/src/controllers`

Recommendation

Update the in-line comments properly, apply the type conversion consistently, and improve the code readability.

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.2 KS-SRK-O-02 Outdated Dependencies

Description

The `cargo audit` (v0.21.0) tool identified 1 vulnerability on dependencies as below.

Crate	Title	RUSTSEC ID	Solution
hashbrown	Borsh serialization of HashMap is non-canonical	RUSTSEC-2024-0402	Upgrade to >=0.15.1

The `npm audit` (v9.5.1) tool identified 16 vulnerabilities (4 moderate, 12 high) on node_modules as below.

Module	Title	Severity
@babel/runtime <7.26.10	Babel has inefficient RexExp complexity in generated code with .replace when transpiling named capturing groups	Moderate
axios 1.0.0 - 1.8.1	xios Requests Vulnerable To Possible SSRF and Credential Leakage via Absolute URL	High
bigint-buffer	bigint-buffer Vulnerable to Buffer Overflow via toBigIntLE() Function	High
nanoid <3.3.8	Predictable results in nanoid generation when given non-integer values	Moderate
serialize-javascript 6.0.0 - 6.0.1	Cross-site Scripting (XSS) in serialize-javascript	Moderate

Evidence

```
[[package]]
name = "hashbrown"
version = "0.13.2"
```

`Location: vaults/Cargo.lock`

Affected Resources

- `vaults`

Recommendation

Keep external dependencies up to date.

References

- https://rustsec.org/advisories/RUSTSEC-2024-0402
- https://github.com/advisories/GHSA-3gc7-fjrx-p6mg

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.3 KS-SRK-O-03 Latency Mitigation

Description

According to the Pyth network, the latency between on-chain oracles and off-chain sources should be accounted for. If adversaries see price changes a short time before the protocol does, they may be able to make profits from the latency.

Affected Resources

- `programs/vaults/src/controllers/token_price.rs`

Recommendation

The Pyth network recommends the following mitigations to reduce the impact of oracle latency:

- A delay can be introduced between the time of the order creation and the time of execution. This delay gives the protocol time to observe price changes and reject trades/transactions that profit over latency.
- Users may be enforced to hold times on positions before they can trade or close to prevent them from exploiting oracle latency.

Note that these mitigations are provided solely for informational purposes. For details, see Reference below.

Reference

- https://docs.pyth.network/price-feeds/best-practices#latency

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.4 KS-SRK-O-04 Lack of Unit Test Vectors

Description

According to the `cargo llvm-cov` tool (v0.6.10), the overall test coverage of code in scope reaches less than 3%. If the test coverage is too low, hidden vulnerabilities may be introduced without being detected when the code base is updated.

Evidence

Filename	Function Coverage	Line Coverage	Region Coverage	Branch Coverage
jupiter/src/lib.rs	33.33% (1/3)	33.33% (1/3)	33.33% (1/3)	- (0/0)
vaults/src/controllers/deposit_token.rs	0.00% (0/1)	0.00% (0/16)	0.00% (0/9)	- (0/0)
vaults/src/controllers/token.rs	0.00% (0/3)	0.00% (0/59)	0.00% (0/9)	- (0/0)
vaults/src/controllers/token_account.rs	0.00% (0/1)	0.00% (0/23)	0.00% (0/1)	- (0/0)
vaults/src/controllers/token_metadata.rs	0.00% (0/1)	0.00% (0/40)	0.00% (0/4)	- (0/0)
vaults/src/controllers/token_price.rs	0.00% (0/2)	0.00% (0/22)	0.00% (0/16)	- (0/0)
vaults/src/controllers/vault_nav.rs	0.00% (0/2)	0.00% (0/41)	0.00% (0/31)	- (0/0)
vaults/src/controllers/vtoken.rs	0.00% (0/4)	0.00% (0/51)	0.00% (0/16)	- (0/0)
vaults/src/controllers/whitelist.rs	0.00% (0/1)	0.00% (0/9)	0.00% (0/7)	- (0/0)
vaults/src/controllers/withdraw_token.rs	0.00% (0/5)	0.00% (0/183)	0.00% (0/69)	- (0/0)
vaults/src/instructions/add_token.rs	0.00% (0/3)	0.00% (0/26)	0.00% (0/6)	- (0/0)
vaults/src/instructions/create_vault.rs	0.00% (0/3)	0.00% (0/65)	0.00% (0/8)	- (0/0)

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

vaults/src/instructions/deposit.rs	0.00% (0/3)	0.00% (0/75)	0.00% (0/20)	- (0/0)
vaults/src/instructions/initialize_whitelist.rs	0.00% (0/3)	0.00% (0/16)	0.00% (0/6)	- (0/0)
vaults/src/instructions/remove_token.rs	0.00% (0/1)	0.00% (0/13)	0.00% (0/4)	- (0/0)
vaults/src/instructions/swap_on_jupiter.rs	0.00% (0/5)	0.00% (0/62)	0.00% (0/16)	- (0/0)
vaults/src/instructions/update_vault_fees.rs	0.00% (0/3)	0.00% (0/38)	0.00% (0/6)	- (0/0)
vaults/src/instructions/withdraw.rs	0.00% (0/3)	0.00% (0/53)	0.00% (0/8)	- (0/0)
vaults/src/jupiter.rs	0.00% (0/1)	0.00% (0/3)	0.00% (0/1)	- (0/0)
vaults/src/lib.rs	9.09% (1/11)	2.04% (1/49)	9.09% (1/11)	- (0/0)
vaults/src/state/events.rs	0.00% (0/14)	0.00% (0/14)	0.00% (0/14)	- (0/0)
vaults/src/state/vault.rs	0.00% (0/8)	0.00% (0/96)	0.00% (0/50)	- (0/0)
vaults/src/state/whitelist.rs	0.00% (0/16)	0.00% (0/72)	0.00% (0/46)	- (0/0)
Totals	**2.06% (2/97)**	**0.19% (2/1029)**	**0.55% (2/361)**	**- (0/0)**

Affected Resources

- `Vaults/program`

Recommendation

Add more test vectors and increase the unit test coverage.

Reference

None

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

4.5 KS-SRK-O-05 Missing Countermeasure Against Price Manipulation

Description

The price of token is determined by the price/data from Pyth Network. However, there is no lower/upper limit of price, compared with the previous price. This is risky due to the possibility of market/oracle price manipulation by attackers. The price from oracle is abnormally high or low, leading the incorrect estimation of the price of token.

Evidence

```
    pub fn get_token_price_from_pyth_feed<'info>(
    price_feed_id: &str,
    price_update: &Account<'info, PriceUpdateV2>,
) -> Result<Price> {
    let feed_id: [u8; 32] = get_feed_id_from_hex(&price_feed_id)?;
    let price = price_update.get_price_no_older_than(
        &Clock::get()?,
        PYTH_PRICE_FEED_MAX_AGE_SECONDS,
        &feed_id,
    )?;

    Ok(price)
}
```

Location: `programs/vaults/src/controllers/token_price.rs`

Affected Resources

- `programs/vaults/src/controllers/token_price.rs`

Recommendation

As shown in the reference, oracle attacks result in incorrect or invalid data being reported that doesn't reflect true asset pricing. Even though the Oracle is robust, there is a possibility that the oracle price data is misreported due to malicious or negligent behavior, or the oracle's data source being compromised, leading to the inaccurate evaluation of token price.

Reference

- Oracle Manipulation Attacks are Rising, Creating a Unique Concern for DeFi

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

5. METHODOLOGY

For this engagement, Kudelski Security used a methodology that is described at a high level in this chapter. This is broken up into the following phases.

5.1 Kickoff

The Kudelski Security Team set up a kickoff meeting where project stakeholders were gathered to discuss the project as well as the responsibilities of participants. During this meeting, we verified the scope of the engagement and discussed the project activities.

5.2 Ramp-up

Ramp-up consisted of the activities necessary to gain proficiency on the particular project. This included the steps required for gaining familiarity with the codebase and technological innovations utilized.

5.3 Review

The review phase is where most of the work on the engagement was performed. In this phase we have analyzed the project for flaws and issues that could impact the security posture. The review for this project was performed using manual methods and utilizing the experience of the reviewer. No dynamic testing was performed, only the use of custom-built scripts and tools was used to assist the reviewer during the testing. We discuss our methodology in more detail in the following subsections.

Code Review

Kudelski Security Team reviewed the code within the project utilizing an appropriate IDE. During every review, the team spends considerable time working with the client to determine correct and expected functionality, business logic, and content, to ensure that findings incorporate this business logic into each description and impact. Following this discovery phase, the team works through the following categories:

- authentication (*e.g.* A07:2021, CWE-306)
- authorization and access control (*e.g.* A01:2021, CWE-862)
- auditing and logging (*e.g.* A09:2021)
- injection and tampering (*e.g.* A03:2021, CWE-20)
- configuration issues (*e.g.* A05:2021, CWE-798)
- logic flaws (*e.g.* A04:2021, CWE-190)
- cryptography (*e.g.* A02:2021)

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

These categories incorporate common weaknesses and vulnerabilities such as the OWASP Top 10 and MITRE Top 25.

Smart Contracts

We reviewed the smart contracts, checking for additional specific issues that can arise such as:

- risk of centralization
- reentrancy
- (non)-adherence to existing standards
- unsafe arithmetic operations

5.4 Reporting

Kudelski Security delivered to the Client a preliminary report in PDF format that contained an executive summary, technical details, and observations about the project.

In the report we not only point out security issues identified but also observations for improvement. The findings are categorized into several buckets, according to their overall severity: **Critical**, **High**, **Medium**, **Low**.

Observations are considered to be **Informational**. Observations can also consist of code review, issues identified during the code review that are not security related, but are general best practices and steps, that can be taken to lower the attack surface of the project.

The technical details are aimed more at developers, describing the issues, the severity ranking and recommendations for mitigation.

5.5 Verify

After the preliminary findings have been delivered, we verify the fixes applied by the Client. After these fixes were verified, we updated the status of the finding in the report.

The output of this phase is the final report with any mitigated findings noted.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

6. VULNERABILITY SCORING SYSTEM

Kudelski Security utilizes a custom approach when computing the vulnerability score, based primarily on the **Impact** of the vulnerability and **Likelihood** of an attack.

Each metric is assigned a ranking of either low, medium or high, based on the criteria defined below. The overall severity score is then computed as described in the next section.

Severity

Severity is the overall score of the finding, weakness or vulnerability as computed from Impact and Likelihood. Other factors, such as availability of tools and exploits, number of instances of the vulnerability and ease of exploitation might also be taken into account when computing the final severity score.

IMPACT / LIKELIHOOD	LOW	MEDIUM	HIGH
HIGH	MEDIUM	HIGH	HIGH
MEDIUM	LOW	MEDIUM	HIGH
LOW	LOW	LOW	MEDIUM

Compute overall severity from Impact and Likelihood. The final severity factor might vary depending on a project's specific context and risk factors.

- **Critical** The identified issue may be immediately exploitable, causing a strong and major negative impact system-wide. They should be urgently remediated or mitigated.
- **High** The identified issue may be directly exploitable causing an immediate negative impact on the users, data, and availability of the system for multiple users.
- **Medium** The identified issue is not directly exploitable but combined with other vulnerabilities may allow for exploitation of the system or exploitation may affect singular users. These findings may also increase in severity in the future as techniques evolve.
- **Low** The identified issue is not directly exploitable but raises the attack surface of the system. This may be through leaking information that an attacker can use to increase the accuracy of their attacks.
- **Informational** findings are best practice steps that can be used to harden the application and improve processes. Informational findings are not assigned a severity score and are classified as Informational instead.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

Impact

The overall effect of the vulnerability against the system or organization based on the areas of concern or affected components discussed with the client during the scoping of the engagement.

- **High** The vulnerability has a severe effect on the company and systems or has an effect within one of the primary areas of concern noted by the client.
- **Medium** It is reasonable to assume that the vulnerability would have a measurable effect on the company and systems that may cause minor financial or reputational damage.
- **Low** There is little to no effect from the vulnerability being compromised. These vulnerabilities could lead to complex attacks or create footholds used in more severe attacks.

Likelihood

The likelihood of an attacker discovering a vulnerability, exploiting it, and obtaining a foothold varies based on a variety of factors including compensating controls, location of the application, availability of commonly used exploits, difficulty of exploitation and institutional knowledge.

- **High** It is extremely likely that this vulnerability will be discovered and abused.
- **Medium** It is likely that this vulnerability will be discovered and abused by a skilled attacker.
- **Low** It is unlikely that this vulnerability will be discovered or abused when discovered.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

7. CONCLUSION

The objective of this code review was to evaluate the overall security of the code base and identify any vulnerabilities that would put the product at risk.

The Kudelski Security Team identified 6 security issues: 1 medium risk and 5 low risks. On average, the effort needed to mitigate these risks is estimated as `low`.

In order to mitigate the risks posed by this engagement's findings, the Kudelski Security Team recommends applying the following best practices:

- Reload the output_`token_account after` the swap and calculate the difference between the pre-swap and post-swap balances to determine the amount of tokens received.
- Implement an emergency shutdown mechanism which could reduce the risk from a serious security breach.
- Modify the vault creation process to include additional unique seeds (e.g., vault name) or require a name registry check.

The Client addressed or acknowledged all these vulnerabilities and observations in the follow-up revision of the codebase.

Kudelski Security remains at your disposal should you have any questions or need further assistance.

Kudelski Security would like to thank StaRKe LLC for their trust, help and support over the course of this engagement and is looking forward to cooperating in the future.

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

8. REFERENCE

- vaults/README.md
- Token Integration with Anchor
- Token-2022 Program

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

9. APPENDIX

9.1 Cargo Audit Log

```
Crate:     curve25519-dalek
Version:   3.2.1
Title:     Timing variability in `curve25519-dalek`'s `Scalar29::sub`/`Scalar52::sub`
Date:      2024-06-18
ID:        RUSTSEC-2024-0344
URL:       https://rustsec.org/advisories/RUSTSEC-2024-0344
Solution:  Upgrade to >=4.1.3

Crate:     ed25519-dalek
Version:   1.0.1
Title:     Double Public Key Signing Function Oracle Attack on `ed25519-dalek`
Date:      2022-06-11
ID:        RUSTSEC-2022-0093
URL:       https://rustsec.org/advisories/RUSTSEC-2022-0093
Solution:  Upgrade to >=2

Crate:     hashbrown
Version:   0.15.0
Title:     Borsh serialization of HashMap is non-canonical
Date:      2024-10-11
ID:        RUSTSEC-2024-0402
URL:       https://rustsec.org/advisories/RUSTSEC-2024-0402
Solution:  Upgrade to >=0.15.1

Crate:     atty
Version:   0.2.14
Warning:   unmaintained
Title:     `atty` is unmaintained
Date:      2024-09-25
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

```
ID:         RUSTSEC-2024-0375
URL:        https://rustsec.org/advisories/RUSTSEC-2024-0375

Crate:      derivative
Version:    2.2.0
Warning:    unmaintained
Title:      `derivative` is unmaintained; consider using an alternative
Date:       2024-06-26
ID:         RUSTSEC-2024-0388
URL:        https://rustsec.org/advisories/RUSTSEC-2024-0388

Crate:      paste
Version:    1.0.15
Warning:    unmaintained
Title:      paste - no longer maintained
Date:       2024-10-07
ID:         RUSTSEC-2024-0436
URL:        https://rustsec.org/advisories/RUSTSEC-2024-0436

Crate:      atty
Version:    0.2.14
Warning:    unsound
Title:      Potential unaligned read
Date:       2021-07-04
ID:         RUSTSEC-2021-0145
URL:        https://rustsec.org/advisories/RUSTSEC-2021-0145

Crate:      borsh
Version:    0.9.3
Warning:    unsound
Title:      Parsing borsh messages with ZST which are not-copy/clone is unsound
Date:       2023-04-12
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

```
ID:         RUSTSEC-2023-0033
URL:        https://rustsec.org/advisories/RUSTSEC-2023-0033

error: 3 vulnerabilities found!
warning: 5 allowed warnings found
```

9.2 Npm Audit Log

```
# npm audit report

@babel/runtime  <7.26.10
Severity: moderate
Babel has inefficient RexExp complexity in generated code with .replace when transpiling named capturing groups - https://github.com/advisories/GHSA-968p-4wvh-cqc8
fix available via `npm audit fix`
node_modules/@babel/runtime

axios  1.0.0 - 1.8.1
Severity: high
axios Requests Vulnerable To Possible SSRF and Credential Leakage via Absolute URL - https://github.com/advisories/GHSA-jr5f-v2jv-69x6
fix available via `npm audit fix`
node_modules/axios

bigint-buffer  *
Severity: high
bigint-buffer Vulnerable to Buffer Overflow via toBigIntLE() Function - https://github.com/advisories/GHSA-3gc7-fjrx-p6mg
No fix available
node_modules/bigint-buffer
  @solana/buffer-layout-utils  *
  Depends on vulnerable versions of bigint-buffer
  node_modules/@solana/buffer-layout-utils
    @solana/spl-token  >=0.2.0-alpha.0
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

```
    Depends on vulnerable versions of @solana/buffer-layout-utils
    Depends on vulnerable versions of @solana/spl-token-group
    Depends on vulnerable versions of @solana/spl-token-metadata
    Depends on vulnerable versions of @solana/web3.js
    node_modules/@solana/spl-token
  @solana/web3.js  1.43.1 - 1.98.0
  Depends on vulnerable versions of bigint-buffer
  node_modules/@solana/web3.js
  node_modules/jito-ts/node_modules/@solana/web3.js
    @coral-xyz/anchor  *
    Depends on vulnerable versions of @coral-xyz/borsh
    Depends on vulnerable versions of @solana/web3.js
    node_modules/@coral-xyz/anchor
    node_modules/@pythnetwork/pyth-solana-
receiver/node_modules/@coral-xyz/anchor
    node_modules/@pythnetwork/solana-utils/node_modules/@coral-
xyz/anchor
      @pythnetwork/solana-utils  *
      Depends on vulnerable versions of @coral-xyz/anchor
      Depends on vulnerable versions of @solana/web3.js
      Depends on vulnerable versions of jito-ts
      node_modules/@pythnetwork/pyth-solana-
receiver/node_modules/@pythnetwork/solana-utils
      node_modules/@pythnetwork/solana-utils
    @coral-xyz/borsh  >=0.26.0
    Depends on vulnerable versions of @solana/web3.js
    node_modules/@coral-xyz/borsh
    node_modules/@pythnetwork/pyth-solana-
receiver/node_modules/@coral-xyz/borsh
    node_modules/@pythnetwork/solana-utils/node_modules/@coral-
xyz/borsh
    @pythnetwork/pyth-solana-receiver  *
    Depends on vulnerable versions of @coral-xyz/anchor
    Depends on vulnerable versions of @pythnetwork/solana-utils
    Depends on vulnerable versions of @solana/web3.js
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

```
    node_modules/@pythnetwork/pyth-solana-receiver
    @solana/spl-token-group  *
    Depends on vulnerable versions of @solana/web3.js
    node_modules/@solana/spl-token-group
    @solana/spl-token-metadata  *
    Depends on vulnerable versions of @solana/web3.js
    node_modules/@solana/spl-token-metadata
    jito-ts  *
    Depends on vulnerable versions of @solana/web3.js
    node_modules/jito-ts

nanoid  <3.3.8
Severity: moderate
Predictable results in nanoid generation when given non-integer
values - https://github.com/advisories/GHSA-mwcw-c2x4-8c55
fix available via `npm audit fix --force`
Will install mocha@11.1.0, which is a breaking change
node_modules/nanoid
  mocha  8.2.0 - 10.5.2
  Depends on vulnerable versions of nanoid
  Depends on vulnerable versions of serialize-javascript
  node_modules/mocha

serialize-javascript  6.0.0 - 6.0.1
Severity: moderate
Cross-site Scripting (XSS) in serialize-javascript -
https://github.com/advisories/GHSA-76p7-773f-r4q5
fix available via `npm audit fix --force`
Will install mocha@11.1.0, which is a breaking change
node_modules/serialize-javascript

16 vulnerabilities (4 moderate, 12 high)
```

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential

DOCUMENT RECIPIENTS

NAME	POSITION	CONTACT INFORMATION
Oscar Garcia Falcon	CEO	oscar@starke.finance
Sounak Pradhan	FullStack Developer	sounak@starke.finance

KUDELSKI SECURITY CONTACTS

NAME	POSITION	CONTACT INFORMATION
Jean-Sebastien Nahon	Application and Blockchain Security Practice Manager	jean-sebastien.nahon@kudelskisecurity.com
Ana Acero	Project Manager/ Operations Coordinator	ana.acero@kudelskisecurity.com

DOCUMENT HISTORY

VERSION	DATE	STATUS/ COMMENTS
V1.0	16 April 2025	Initial draft
V1.1	2 May 2025	Final proposal

© 2025 Nagravision Sàrl / All Rights Reserved
Confidential